



INSTITUT
TEKNOLOGI
HARAPAN
BANGSA
Veritas vos liberabit

MODUL PRAKTIKUM

Teknik Informatika

IF-9P2

Pemrograman Piranti Bergerak

Firhat Hidayat, S.Kom., M.T.

2019 © All rights reserved.

Dilarang memperbanyak dan/atau mengcopy sebagian atau seluruh material
dalam dokumen ini tanpa persetujuan tertulis dari Institut Teknologi Harapan Bangsa

MODUL 3

Fragment

1. Objective

Codelab ini menitikberatkan pada implementasi fragment pada activity

2. Activities

Codelab selanjutnya adalah dengan membuat aplikasi yang dapat menerapkan kegunaan Fragment dengan fungsi sebagai berikut

1. Membuat tampilan fleksibel dengan fragment,
2. Membuat tampilan yang dapat digeser-geser, dan
3. Membuat tampilan yang dapat disesuaikan dengan perubahan orientasi *device*.

3. Exercise

Logika dasar untuk membangun aplikasi,

Melakukan klik ke button → memanggil fragment atau activity dengan atau tanpa data
→ menampilkan activity atau fragment yang dituju

3.1 Basic Fragment

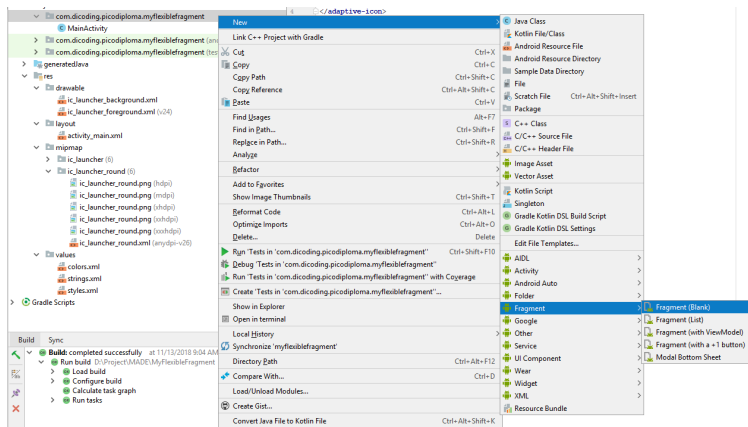
1. Buat Project baru di Android Studio dengan kriteria sebagai berikut:

Nama Project	MyFlexibleFragment
Target & Minimum Target SDK	Phone and Tablet, Api level 21
Tipe Activity	Empty Activity
Activity Name	MainActivity
Use AndroidX artifacts	TRUE
Language	Kotlin / Java

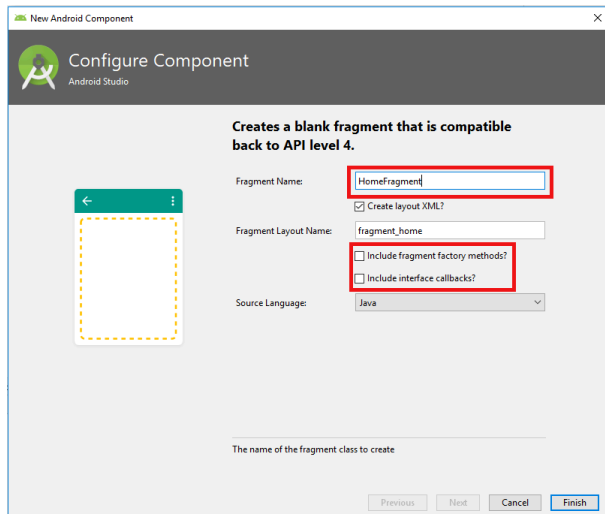
2. Selanjutnya kita akan membangun antarmuka (*interface*) seperti ini: Pada **activity_main.xml**, silakan kondisikan kode pada berkas tersebut menjadi seperti berikut:

1. `<?xml version="1.0" encoding="utf-8"?>`
2. `<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"`
3. `xmlns:tools="http://schemas.android.com/tools"`
4. `android:id="@+id/frame_container"`
5. `android:layout_width="match_parent"`
6. `android:layout_height="match_parent"/>`

3. Kemudian kita buat beberapa fragment untuk mengimplementasikan perpindahan tampilan tanpa perpindahan activity. Pertama kita buat fragment dengan nama **HomeFragment**. Caranya : klik kanan pada package utama pada proyek aplikasi Anda → **New** → **Fragment** → **Fragment (Blank)**.



4. Setelah tampil dialog untuk fragment, isikan **HomeFragment** pada kolom **Fragment Name** dan **unchecked** untuk kedua pilihan (**include fragment factory methods** dan **include interface callbacks**) seperti gambar di bawah ini. Klik **Finish** untuk melanjutkan penciptaan fragment.



5. Setelah **HomeFragment** tercipta, pada **fragment_home.xml** mari sesuaikan tampilannya dengan menambahkan sebuah obyek textview dan sebuah obyek button seperti berikut:

```

1. <LinearLayout xmlns:android="http://schemas.android.com/apk/res/
   android"
2.     xmlns:tools="http://schemas.android.com/tools"
3.     android:layout_width="match_parent"
4.     android:layout_height="match_parent"
5.     android:orientation="vertical">
6.
7.     <TextView
8.         android:layout_width="match_parent"
9.         android:layout_height="wrap_content"
10.        android:layout_margin="16dp"
11.        android:text="@string/hello_home_fragment" />
12.
13.    <Button
14.        android:id="@+id/btn_category"
15.        android:layout_width="match_parent"
16.        android:layout_height="wrap_content"
17.        android:layout_marginLeft="16dp"
18.        android:layout_marginRight="16dp"
19.        android:text="@string/to_category" />
20. </LinearLayout>

```

Akan ada yang *error* pada bagian `android:text` .Seperti pada modul sebelumnya kita perlu menambahkan *resource* string-nya.

Tambahkan beberapa baris kode di bawah ini di dalam *res* → *values* → *strings.xml*.

```
1. <resources>
2.   <string name="app_name">MyFlexibleFragment</string>
3.
4.   <string name="hello_blank_fragment">Hello blank fragment</string>
5.   <string name="this_profile">Ini activity Profile</string>
6.   <string name="this_category">Ini fragment Category</string>
7.   <string name="category_lifestyle">Ke fragment Lifestyle</string>
8.   <string name="category_name">Category Name</string>
9.   <string name="category_description">Category Description</string>
10.  <string name="to_profile">Ke Halaman Profile Activity</string>
11.  <string name="show_dialog">Tampilkan sebuah dialog</string>
12.  <string name="hello_home_fragment">Hello Ini Home Fragment</string>
13.  <string name="to_category">Ke fragment Category</string>
14.  <string name="question_coach">Siapa pelatih tersukses Manchester
    United?</string>
15.  <string name="sir_alex_ferguson">Sir Alex Ferguson</string>
16.  <string name="jose_mourinho">Jose Mourinho</string>
17.  <string name="louis_van_gaal">Louis Van Gaal</string>
18.  <string name="david_moyes">David Moyes</string>
19.  <string name="choose">Pilih</string>
20.  <string name="close">Tutup</string>
21. </resources>
```

Resource String ini akan kita gunakan selama latihan fragment.

6. Pada **HomeFragment** lakukan penyesuaian kode sebagai berikut:

```
1. class HomeFragment : Fragment(), View.OnClickListener {
2.
3.     ...
4.
5.     override fun onCreateView(view: View, savedInstanceState: Bundle?)
6.     {
7.         super.onCreateView(view, savedInstanceState)
8.         val btnCategory:Button = view.findViewById(R.id.btn_category)
9.         btnCategory.setOnClickListener(this)
10.
11.     override fun onClick(v: View) {
12.
13.     }
```

Di sini kita juga siapkan kode *listener* **onClick**. *Listener* di sini akan kita gunakan pada latihan berikutnya.

7. Selanjutnya, pada **MainActivity**, kita tanamkan **HomeFragment** ke dalam activity tersebut sehingga bisa tampil ke layar pengguna dengan menambahkan beberapa baris berikut:

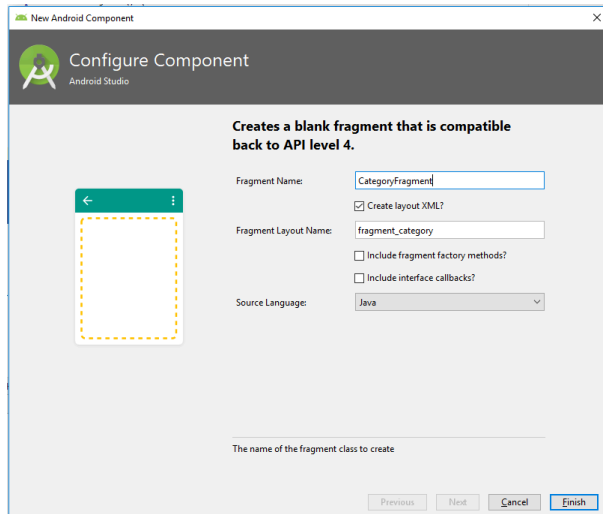
```
1. ...
2. override fun onCreate(savedInstanceState: Bundle?) {
3.     super.onCreate(savedInstanceState)
4.     setContentView(R.layout.activity_main)
5.
6.     val fragmentManager = supportFragmentManager
7.     val mHomeFragment = HomeFragment()
8.     val fragment =
        fragmentManager.findFragmentByTag(HomeFragment::class.java.simpleName)
9.
10.    if (fragment !is HomeFragment) {
11.        Log.d("MyFlexibleFragment", "Fragment Name :" +
            HomeFragment::class.java.simpleName)
12.        fragmentManager
13.            .beginTransaction()
14.            .add(R.id.frame_container, mHomeFragment,
                HomeFragment::class.java.simpleName)
15.            .commit()
16.    }
17. }
18. ...
```

8. Setelah selesai semua, silakan jalankan aplikasi Anda.

3.2 Flexible Fragment di Satu Activity

Selanjutnya kita akan mencoba untuk membuat fragment baru yang fleksibel

1. Dengan cara yang sama seperti sebelumnya, buat fragment baru dengan nama **CategoryFragment** . Jangan lupa untuk *unchecked* pilihan untuk *generate default methods* dan *callbacks*.



1. Pada **fragment_category** silakan kondisikan kode Anda menjadi seperti berikut:

```

1. <LinearLayout xmlns:android="http://schemas.android.com/apk/res/
  android"
2.     xmlns:tools="http://schemas.android.com/tools"
3.     android:layout_width="match_parent"
4.     android:layout_height="match_parent"
5.     android:orientation="vertical"
6.     android:padding="16dp">
7.
8.     <TextView
9.         android:layout_width="match_parent"
10.        android:layout_height="wrap_content"
11.        android:layout_marginBottom="16dp"
12.        android:text="@string/this_category" />
13.
14.    <Button
15.        android:id="@+id/btn_detail_category"
16.        android:layout_width="match_parent"
17.        android:layout_height="wrap_content"
18.        android:text="@string/category_lifestyle" />
19. </LinearLayout>

```

2. Selesai dengan layout xml, kini pada berkas **CategoryFragment** modifikasi kodenya menjadi sebagai berikut:

```

1. class CategoryFragment : Fragment(), View.OnClickListener {
2.     ...
3.     ...
4.     ...
5.     override fun onCreateView(view: View, savedInstanceState: Bundle?)
6.     {
7.         super.onCreateView(view, savedInstanceState)
8.         val btnDetailCategory: Button =
9.         view.findViewById(R.id.btn_detail_category)
10.        btnDetailCategory.setOnClickListener(this)
11.    }
12.    override fun onClick(v: View) {
13.        if (v.id == R.id.btn_detail_category) {
14.            ...
15.        }
16.    }

```

Sama seperti sebelumnya, kita siapkan `onClick` untuk modul berikutnya yaitu **"Mengirim Data Antar Fragment"**.

3. Sekarang kembali pada **HomeFragment**. Tambahkan baris berikut pada metode `onClick()`.

```

1. override fun onClick(v: View) {
2.     if (v.id == R.id.btn_category) {
3.         val mCategoryFragment = CategoryFragment()
4.         val mFragmentManager = fragmentManager as FragmentManager
5.         mFragmentManager
6.             .beginTransaction()
7.             .replace(R.id.frame_container, mCategoryFragment,
8.             CategoryFragment::class.java.simpleName)
9.             .addToBackStack(null)
10.            .commit()
11.    }

```


4. Sehingga kode dari **HomeFragment** menjadi seperti ini:

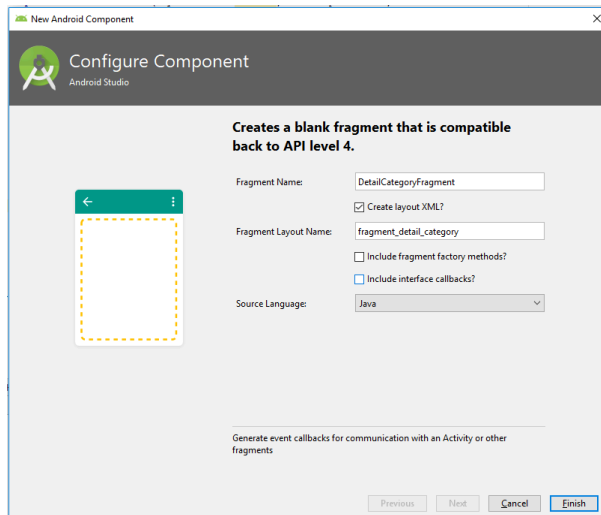
```
1. class HomeFragment : Fragment(), View.OnClickListener {
2.
3.     override fun onCreateView(inflater: LayoutInflater, container:
4. ViewGroup?,
5.                               savedInstanceState: Bundle?): View? {
6.         // Inflate the layout for this fragment
7.         return inflater.inflate(R.layout.fragment_home, container,
8. false)
9.     }
10.
11.     override fun onViewCreated(view: View, savedInstanceState: Bundle?)
12. {
13.         super.onViewCreated(view, savedInstanceState)
14.         val btnCategory:Button = view.findViewById(R.id.btn_category)
15.         btnCategory.setOnClickListener(this)
16.     }
17.
18.     override fun onClick(v: View) {
19.         if (v.id == R.id.btn_category) {
20.             val mCategoryFragment = CategoryFragment()
21.             val mFragmentManager = fragmentManager as FragmentManager
22.             mFragmentManager
23.                 .beginTransaction()
24.                 .replace(R.id.frame_container, mCategoryFragment,
25. CategoryFragment::class.java.simpleName)
26.                 .addToBackStack(null)
27.                 .commit()
28.         }
29.     }
30. }
```

5. Setelah selesai, silakan jalankan aplikasi lagi. Klik tombol **Ke Fragment Category**. Aplikasi sudah berpindah tampilan tanpa berpindah activity.

3.3 Mengirim Data Antar Fragment

Kita telah belajar bagaimana berpindah dari satu fragment ke fragment lain. Nah, kurang lengkap rasanya kalau belum tahu bagaimana berpindah fragment dengan membawa data. Terdapat dua cara yakni dengan menggunakan obyek bundle dan *setter and getter*. Mari langsung praktikkan kedua cara tersebut.

1. Buat fragment lagi dengan nama ***DetailCategoryFragment***.



2. Pada ***fragment_detail_category*** sesuaikan kodenya menjadi seperti berikut:

```
1. <LinearLayout xmlns:android="http://schemas.android.com/apk/res/
   android"
2.     xmlns:tools="http://schemas.android.com/tools"
3.     android:layout_width="match_parent"
4.     android:layout_height="match_parent"
5.     android:orientation="vertical"
6.     android:padding="16dp">
7.
8.     <TextView
9.         android:id="@+id/tv_category_name"
10.        android:layout_width="match_parent"
11.        android:layout_height="wrap_content"
12.        android:layout_marginBottom="16dp"
13.        android:text="@string/category_name" />
14.
15.    <TextView
16.        android:id="@+id/tv_category_description"
17.        android:layout_width="match_parent"
18.        android:layout_height="wrap_content"
19.        android:layout_marginBottom="16dp"
```

```

20.         android:text="@string/category_description" />
21.
22.     <Button
23.         android:id="@+id/btn_profile"
24.         android:layout_width="match_parent"
25.         android:layout_height="wrap_content"
26.         android:text="@string/to_profile" />
27.
28.     <Button
29.         android:id="@+id/btn_show_dialog"
30.         android:layout_width="match_parent"
31.         android:layout_height="wrap_content"
32.         android:text="@string/show_dialog" />
33. </LinearLayout>

```

3. Setelah selesai, silakan lengkapi kode pada **DetailCategoryFragment**. Pertama, tambahkan beberapa *view*, dan juga *casting*-nya.

```

1. class DetailCategoryFragment : Fragment(){
2.
3.     lateinit var tvCategoryName: TextView
4.     lateinit var tvCategoryDescription: TextView
5.     lateinit var btnProfile: Button
6.     lateinit var btnShowDialog: Button
7.
8.     ...
9.
10.    override fun onCreateView(view: View, savedInstanceState: Bundle?)
11.    {
12.        super.onCreateView(view, savedInstanceState)
13.        tvCategoryName = view.findViewById(R.id.tv_category_name)
14.        tvCategoryDescription =
15.            view.findViewById(R.id.tv_category_description)
16.        btnProfile = view.findViewById(R.id.btn_profile)
17.        btnProfile.setOnClickListener(this)
18.        btnShowDialog = view.findViewById(R.id.btn_show_dialog)
19.        btnShowDialog.setOnClickListener(this)
20.    }
21. }

```

Kemudian metode `setOnClickListener` akan mengalami *error* karena Anda belum mengimplementasi `onClick()` ke **DetailCategoryFragment**. Maka implementasikanlah `onClick` di kelas *fragment*-nya dan juga *set listener* di *view*-nya.

```

1. class DetailCategoryFragment : Fragment(), View.OnClickListener {
2.     ...
3.     ...
4.     override fun onClick(v: View) {
5.         when (v.id) {
6.             ...
7.             ...
8.             ...
9.         }
10.    }
11. }

```

Dan tambahkan beberapa variabel dan berikan aksi ketika button diklik.

```

1. class DetailCategoryFragment : Fragment(), View.OnClickListener {
2.     var description: String? = null
3.     companion object {
4.         var EXTRA_NAME = "extra_name"
5.         var EXTRA_DESCRIPTION = "extra_description"
6.     }
7.     ...
8.     ...
9.     ...
10.    ...
11.    override fun onActivityCreated(savedInstanceState: Bundle?) {
12.        super.onActivityCreated(savedInstanceState)
13.        if (savedInstanceState != null) {
14.            val descFromBundle =
15.                savedInstanceState.getString(EXTRA_DESCRIPTION)
16.            description = descFromBundle
17.        }
18.        if (arguments != null) {
19.            val categoryName = arguments?.getString(EXTRA_NAME)
20.            tv_category_name.text = categoryName
21.            tv_category_description.text = description
22.        }
23.    }
24.    override fun onClick(v: View) {
25.        when (v.id) {
26.            R.id.btn_profile -> {
27.                ...
28.            }
29.            R.id.btn_show_dialog -> {
30.                ...
31.            }
32.        }
33.    }
34. }
35. }

```

Kode di atas akan mendemonstrasikan bagaimana melakukan penampilan data yang dikirim melalui perpindahan fragment.

Sehingga DetailCategoryFragment menjadi seperti ini:

```
1. class DetailCategoryFragment : Fragment(), View.OnClickListener {
2.
3.     lateinit var tvCategoryName: TextView
4.     lateinit var tvCategoryDescription: TextView
5.     lateinit var btnProfile: Button
6.     lateinit var btnShowDialog: Button
7.     var description: String? = null
8.
9.     companion object {
10.         var EXTRA_NAME = "extra_name"
11.         var EXTRA_DESCRIPTION = "extra_description"
12.     }
13.
14.     override fun onCreateView(inflater: LayoutInflater, container:
15.         ViewGroup?,
16.                               savedInstanceState: Bundle?): View? {
17.         // Inflate the layout for this fragment
18.         return inflater.inflate(R.layout.fragment_detail_category,
19.                                 container, false)
20.     }
21.
22.     override fun onViewCreated(view: View, savedInstanceState: Bundle?)
23.     {
24.         super.onViewCreated(view, savedInstanceState)
25.         tvCategoryName = view.findViewById(R.id.tv_category_name)
26.         tvCategoryDescription =
27.             view.findViewById(R.id.tv_category_description)
28.         btnProfile = view.findViewById(R.id.btn_profile)
29.         btnProfile.setOnClickListener(this)
30.         btnShowDialog = view.findViewById(R.id.btn_show_dialog)
31.         btnShowDialog.setOnClickListener(this)
32.     }
33.
34.     override fun onActivityCreated(savedInstanceState: Bundle?) {
35.         super.onActivityCreated(savedInstanceState)
36.
37.         if (savedInstanceState != null) {
38.             val descFromBundle =
39.                 savedInstanceState.getString(EXTRA_DESCRIPTION)
40.             description = descFromBundle
41.         }
42.     }
43. }
```

```

38.         if (arguments != null) {
39.             val categoryName = arguments?.getString(EXTRA_NAME)
40.             tv_category_name.text = categoryName
41.             tv_category_description.text = description
42.         }
43.     }
44.
45.     override fun onClick(v: View) {
46.         when (v.id) {
47.             R.id.btn_profile -> {
48.
49.             }
50.
51.             R.id.btn_show_dialog -> {
52.
53.             }
54.         }
55.     }
56. }

```

4. Baik, tinggal selangkah lagi kita menyelesaikan proses implementasi pengiriman data melalui perpindahan fragment. Sekarang buka kembali **CategoryFragment** lalu tambahkan baris berikut pada *method* `onClick()`.

```

1. override fun onClick(v: View) {
2.     if (v.id == R.id.btn_detail_category) {
3.         val mDetailCategoryFragment = DetailCategoryFragment()
4.
5.         val mBundle = Bundle()
6.         mBundle.putString(DetailCategoryFragment.EXTRA_NAME,
7.             "Lifestyle")
8.         val description = "Kategori ini akan berisi produk-produk
9.             lifestyle"
10.        mDetailCategoryFragment.arguments = mBundle
11.        mDetailCategoryFragment.description = description
12.        /*
13.        Method addToBackStack akan menambahkan fragment ke backstack
14.        Behaviour dari back button akan cek fragment dari backstack,
15.        jika ada fragment di dalam backstack maka fragment yang akan di
16.        close / remove
17.        jika sudah tidak ada fragment di dalam backstack maka activity
18.        yang akan di close / finish
19.        */
20.        val mFragmentManager = fragmentManager as FragmentManager
21.        mFragmentManager
22.            .beginTransaction()

```

```

21.         .replace(R.id.frame_container, mDetailCategoryFragment,
DetailCategoryFragment::class.java.simpleName)
22.         .addToBackStack(null)
23.         .commit()
24.     }
25. }

```

Sehingga kode **CategoryFragment** kita sekarang menjadi:

```

1. class CategoryFragment : Fragment(), View.OnClickListener {
2.
3.
4.     override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?,
5.                               savedInstanceState: Bundle?): View? {
6.         // Inflate the layout for this fragment
7.         return inflater.inflate(R.layout.fragment_category, container,
false)
8.     }
9.
10.    override fun onViewCreated(view: View, savedInstanceState: Bundle?)
{
11.        super.onViewCreated(view, savedInstanceState)
12.        val btnDetailCategory: Button =
view.findViewById(R.id.btn_detail_category)
13.        btnDetailCategory.setOnClickListener(this)
14.    }
15.
16.    override fun onClick(v: View) {
17.        if (v.id == R.id.btn_detail_category) {
18.            val mDetailCategoryFragment = DetailCategoryFragment()
19.
20.            val mBundle = Bundle()
21.            mBundle.putString(DetailCategoryFragment.EXTRA_NAME,
"Lifestyle")
22.            val description = "Kategori ini akan berisi produk-produk
lifestyle"
23.
24.            mDetailCategoryFragment.arguments = mBundle
25.            mDetailCategoryFragment.description = description
26.
27.            /*
28.            Method addToBackStack akan menambahkan fragment ke
backstack
29.            Behaviour dari back button akan cek fragment dari
backstack,

```

```

30.         jika ada fragment di dalam backstack maka fragment yang
           akan di close / remove
31.         jika sudah tidak ada fragment di dalam backstack maka
           activity yang akan di close / finish
32.         */
33.         val mFragmentManager = fragmentManager as FragmentManager
34.         mFragmentManager
35.             .beginTransaction()
36.             .replace(R.id.frame_container,
mDetailCategoryFragment, DetailCategoryFragment::class.java.simpleName)
37.             .addToBackStack(null)
38.             .commit()
39.     }
40. }
41. }

```

5. Sekarang setelah selesai semua, silakan jalankan aplikasinya untuk melihat hasil kode yang di atas. Seharusnya ketika Anda mengklik tombol **Kategori Lifestyle** pada **CategoryFragment** akan ada data yang dikirimkan sewaktu perpindahan fragment itu melalui *object* bundle dan mekanisme metode *setter and getter*. Tampilan aplikasi Anda sekarang sudah menjadi seperti file yang terlampir di classroom.